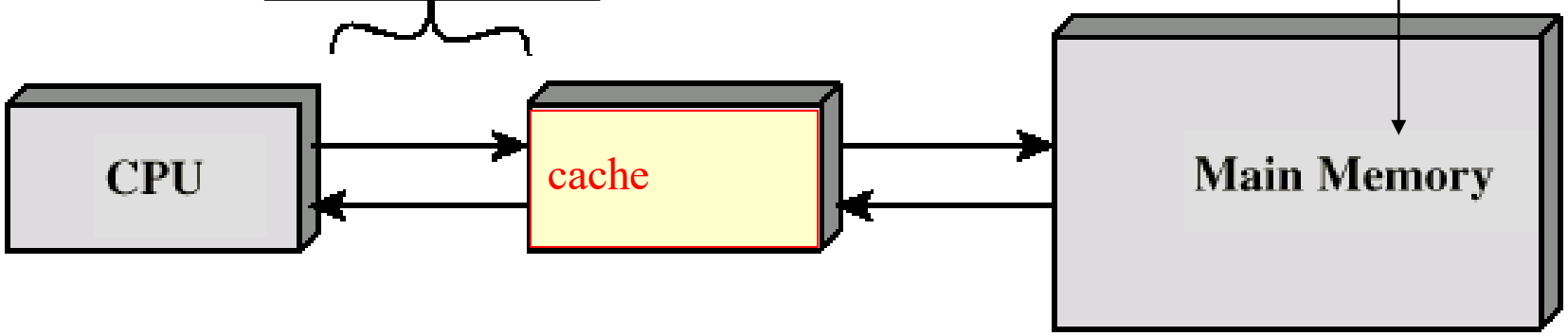
Cache Memory

Cache

smaller than main memory

- small amount of fast memory
- sits between normal main memory and CPU
- may be located on CPU chip or module

cache views main memory as organized in "blocks"



Cache

- The purpose of cache memory is to ***speed up*** data accesses for processor by storing information in faster memory made of SRAM

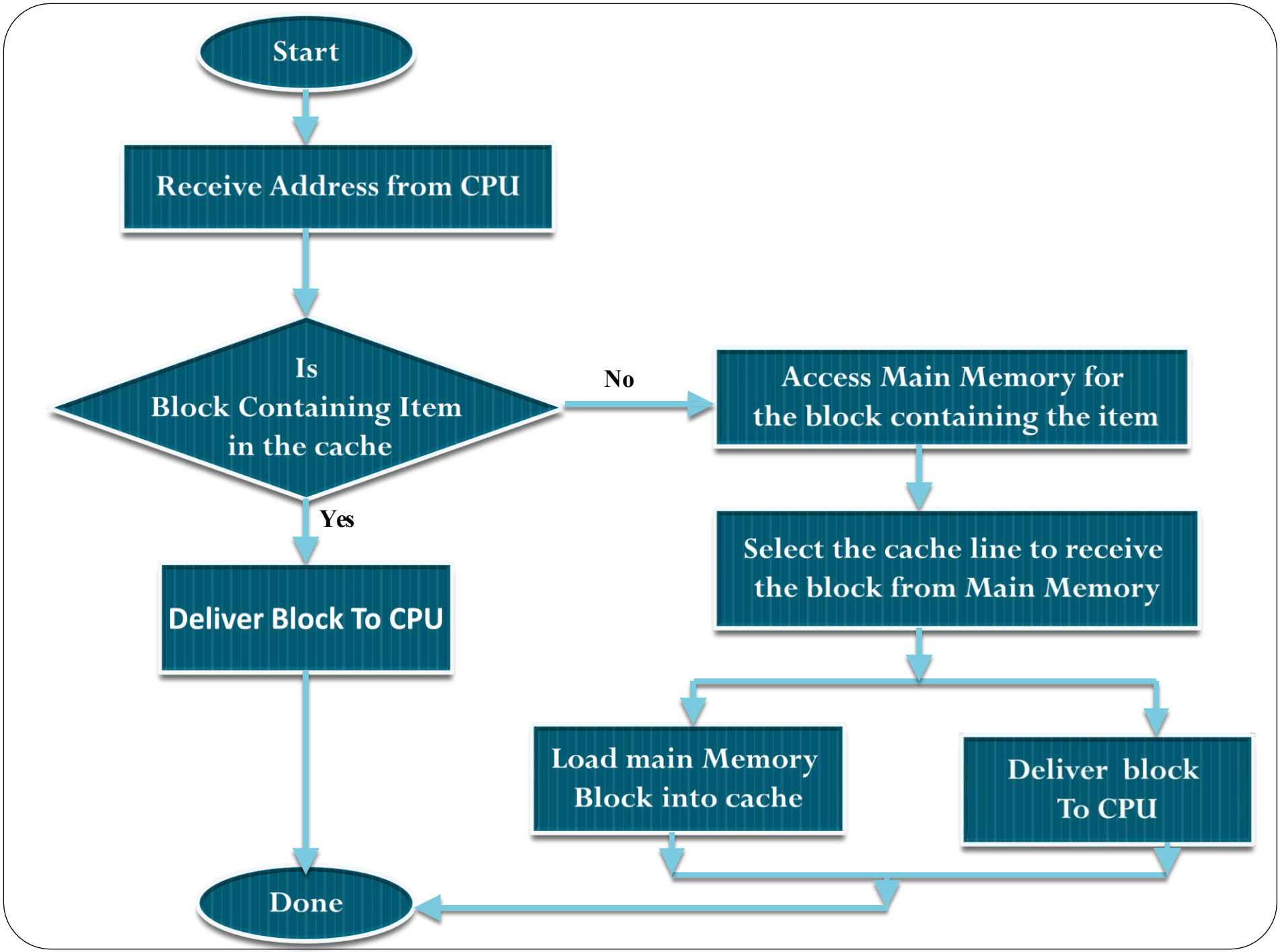
- The data stored in cache is data that the processor is ***likely to use in the very near future***

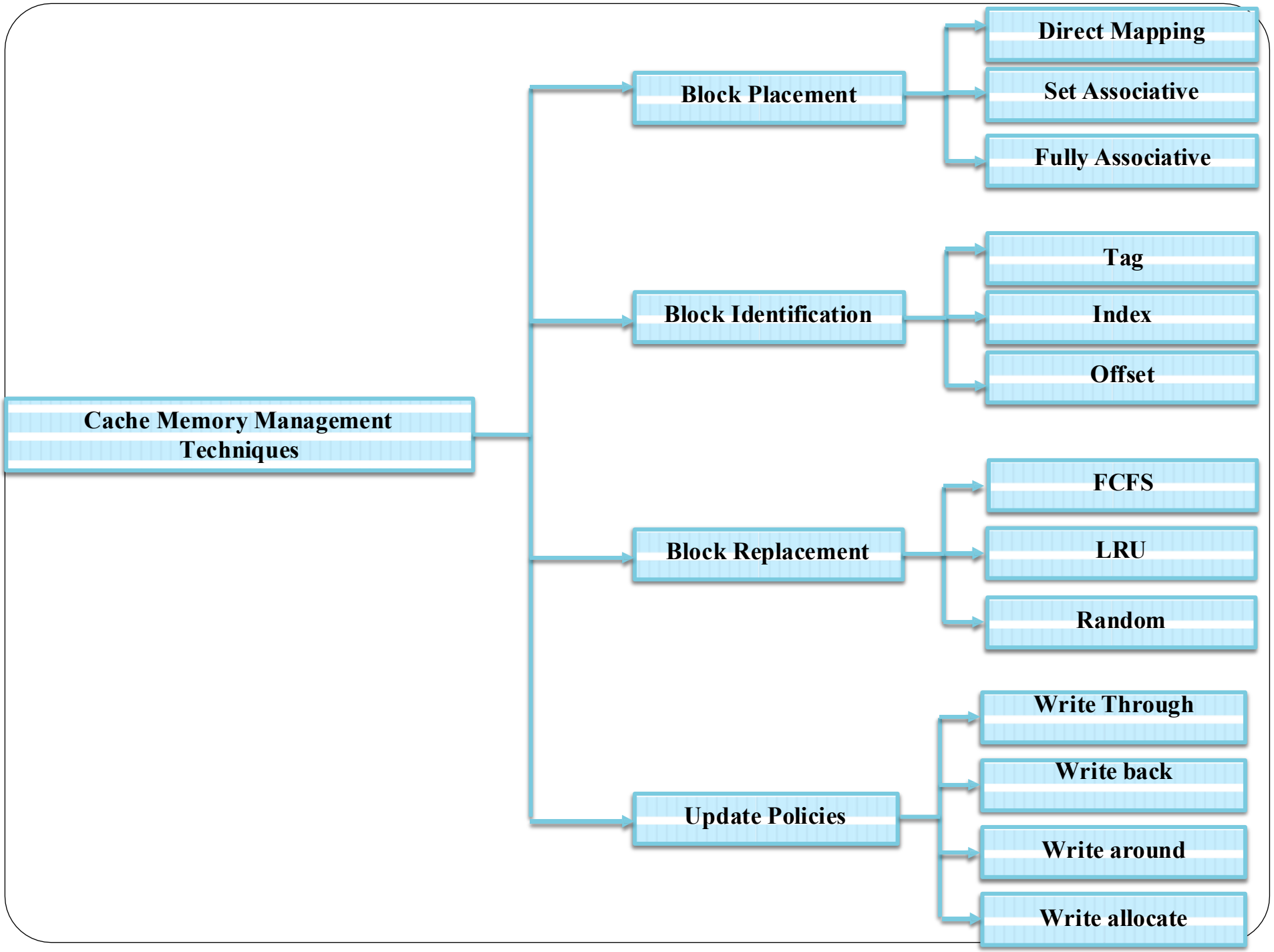
Locality of Reference

- An *implication of locality* is that we can predict with reasonable accuracy what instructions and data a program will use in the near future based on its accesses in the recent past.
- ***Temporal locality***: a recently executed instruction is likely to be executed again soon
- ***Spatial locality***: instructions with addresses close to a recently executed instruction are likely to be executed soon.

Parameters of Cache memory

- Cache Hit
  - A referenced item is found in the cache by the processor
- Cache Miss
  - A referenced item is not present in the cache
- Hit ratio
  - Ratio of number of hits to total number of references => number of hits/(number of hits + number of Miss)
- Miss penalty
  - Additional cycles required to serve the miss
  - Time required for the cache miss depends on both the latency and bandwidth
  - Latency – time to retrieve the first word of the block
  - Bandwidth – time to retrieve the rest of this block


Start
Receive Address from CPU
Is
Block Containing Item
in the cache
No
Access Main Memory for
the block containing the item
Yes
Select the cache line to receive
the block from Main Memory
Deliver Block To CPU
Load main Memory
Block into cache
Deliver block
To CPU
Done


Cache Memory Management Techniques
Block Placement
Direct Mapping
Set Associative
Fully Associative
Block Identification
Tag
Index
Offset
Block Replacement
FCFS
LRU
Random
Update Policies
Write Through
Write back
Write around
Write allocate

Example

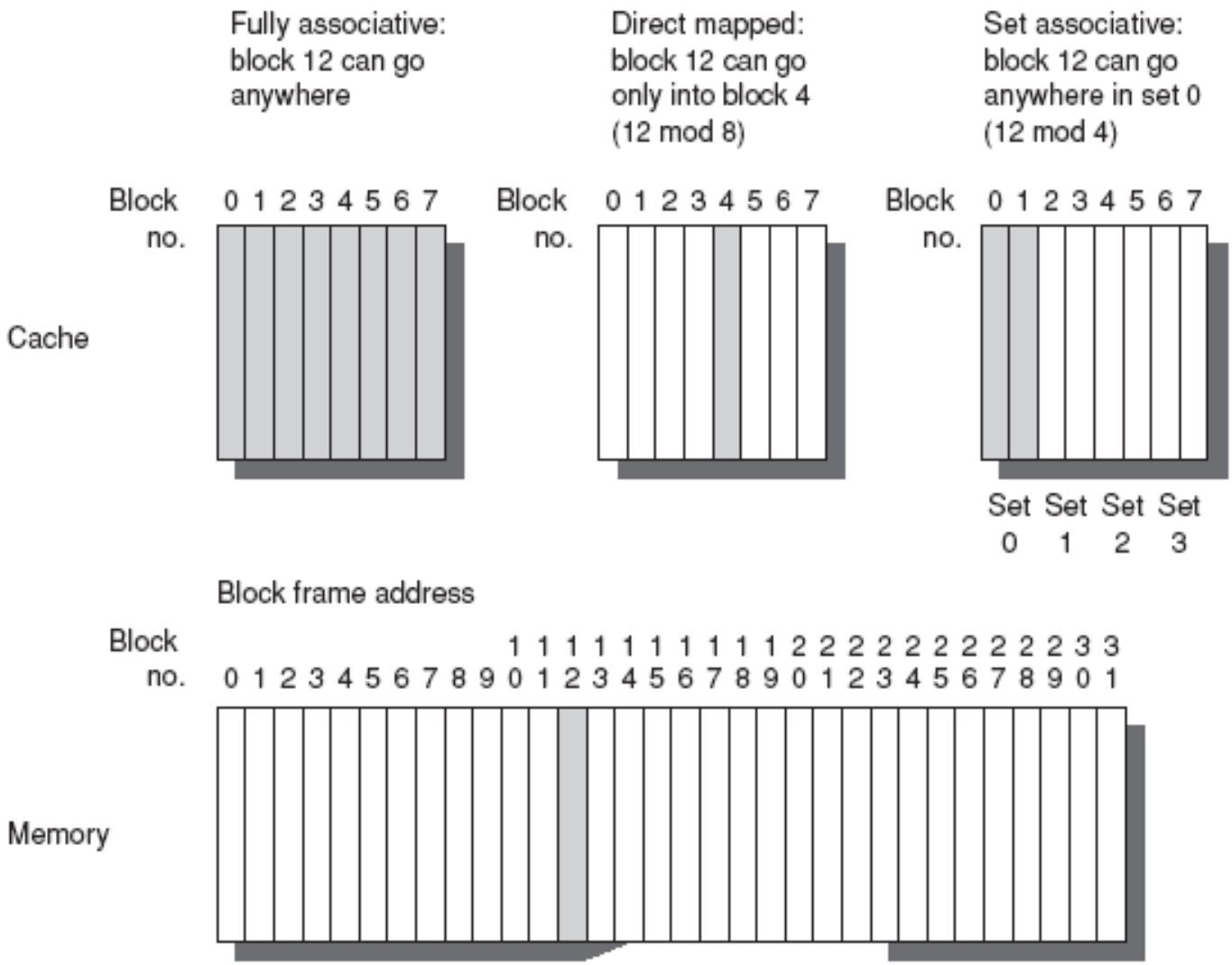
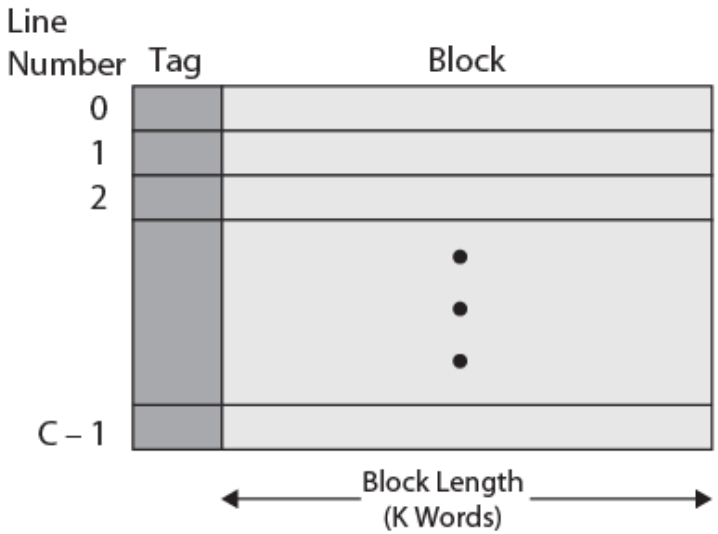
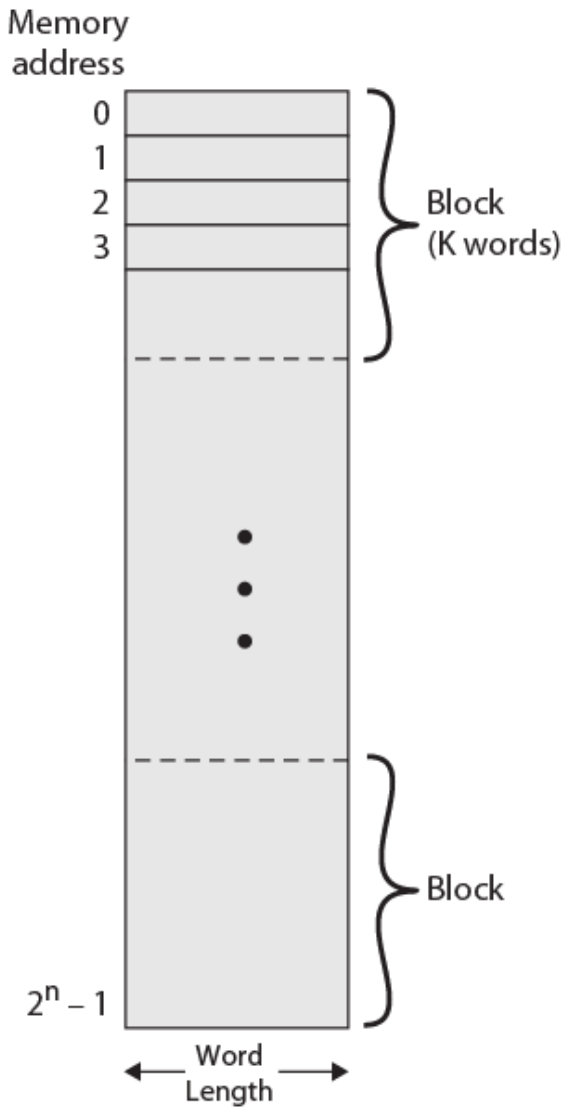


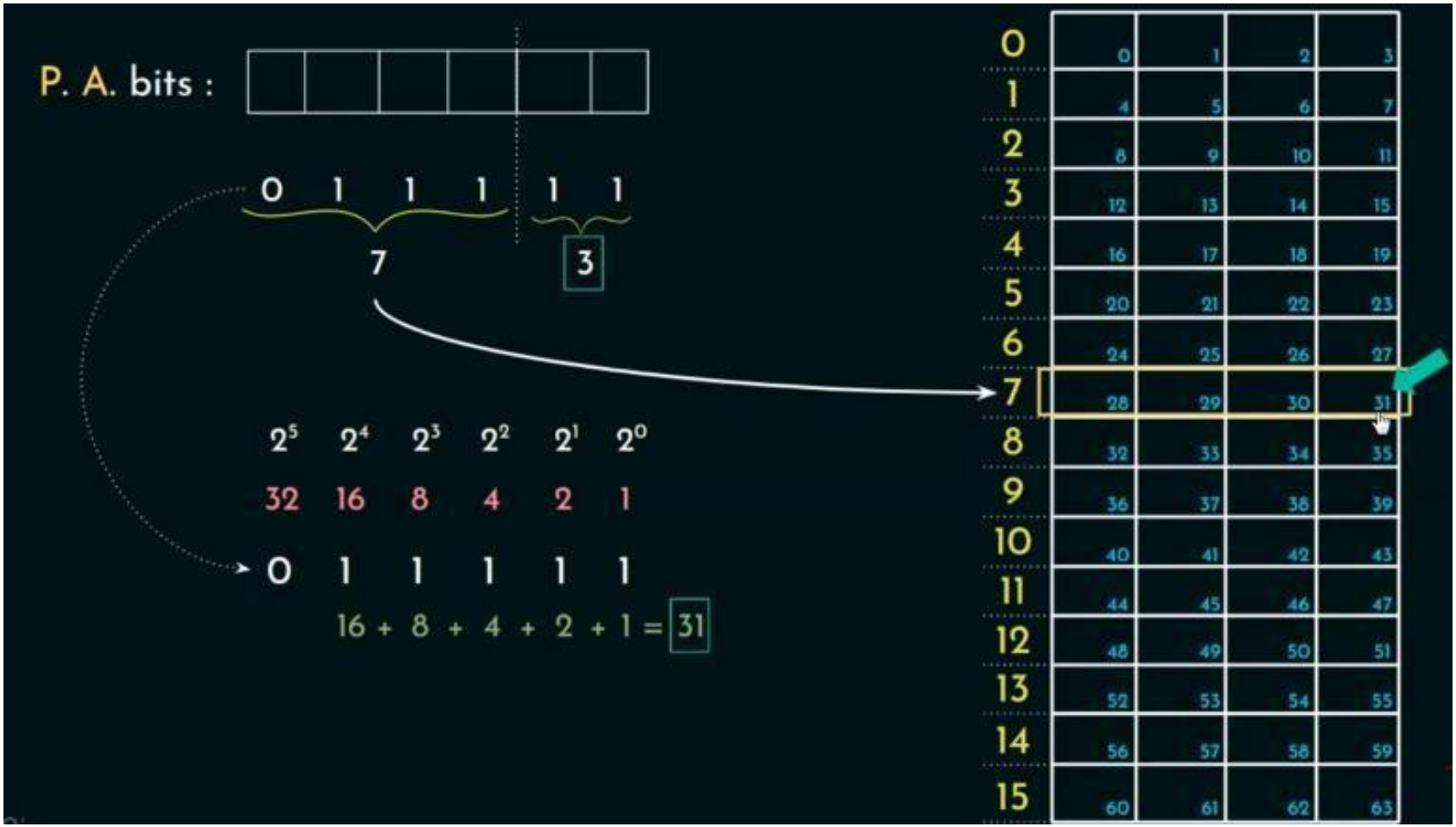
Figure C.2 This example cache has eight block frames and memory has 32 blocks.



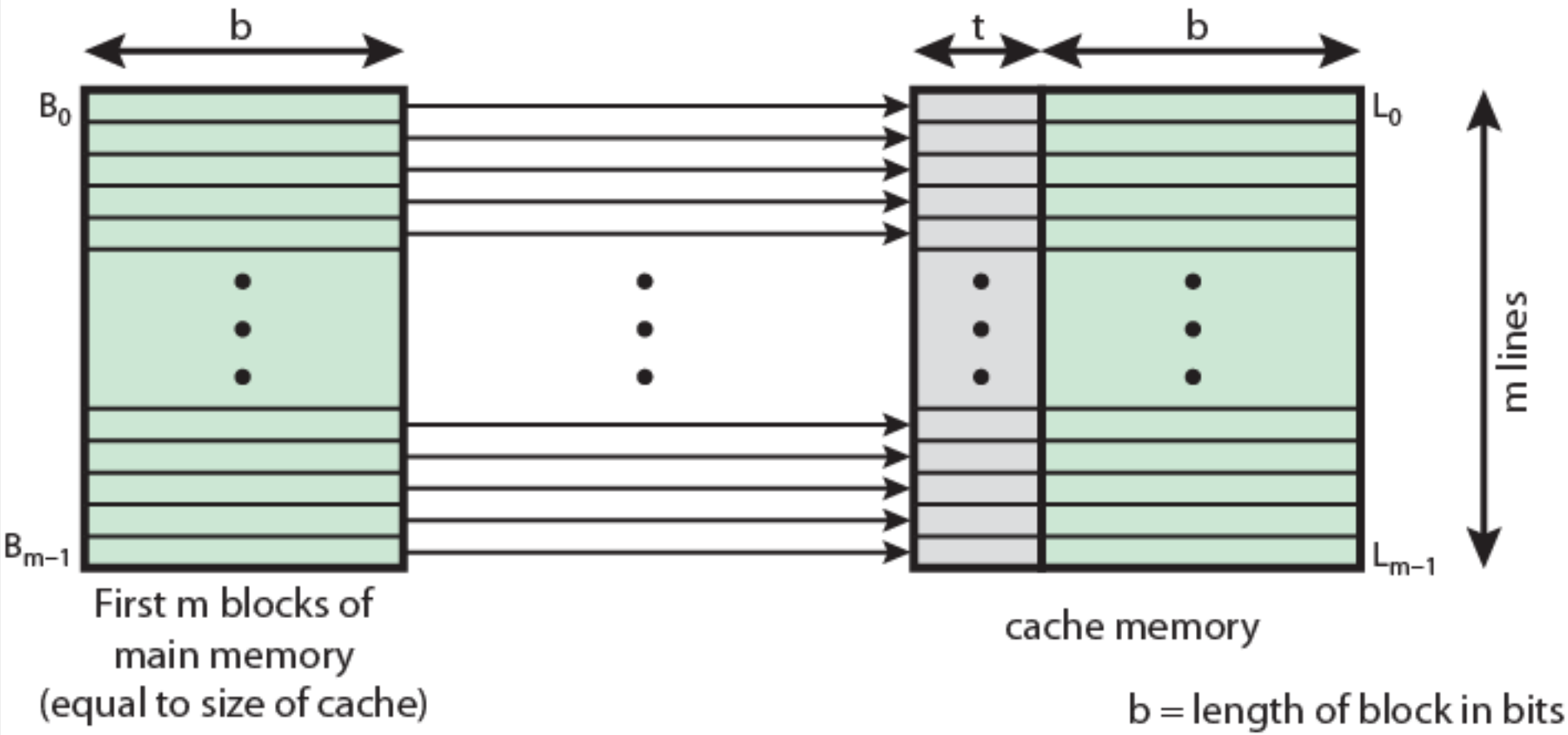
(a) Cache



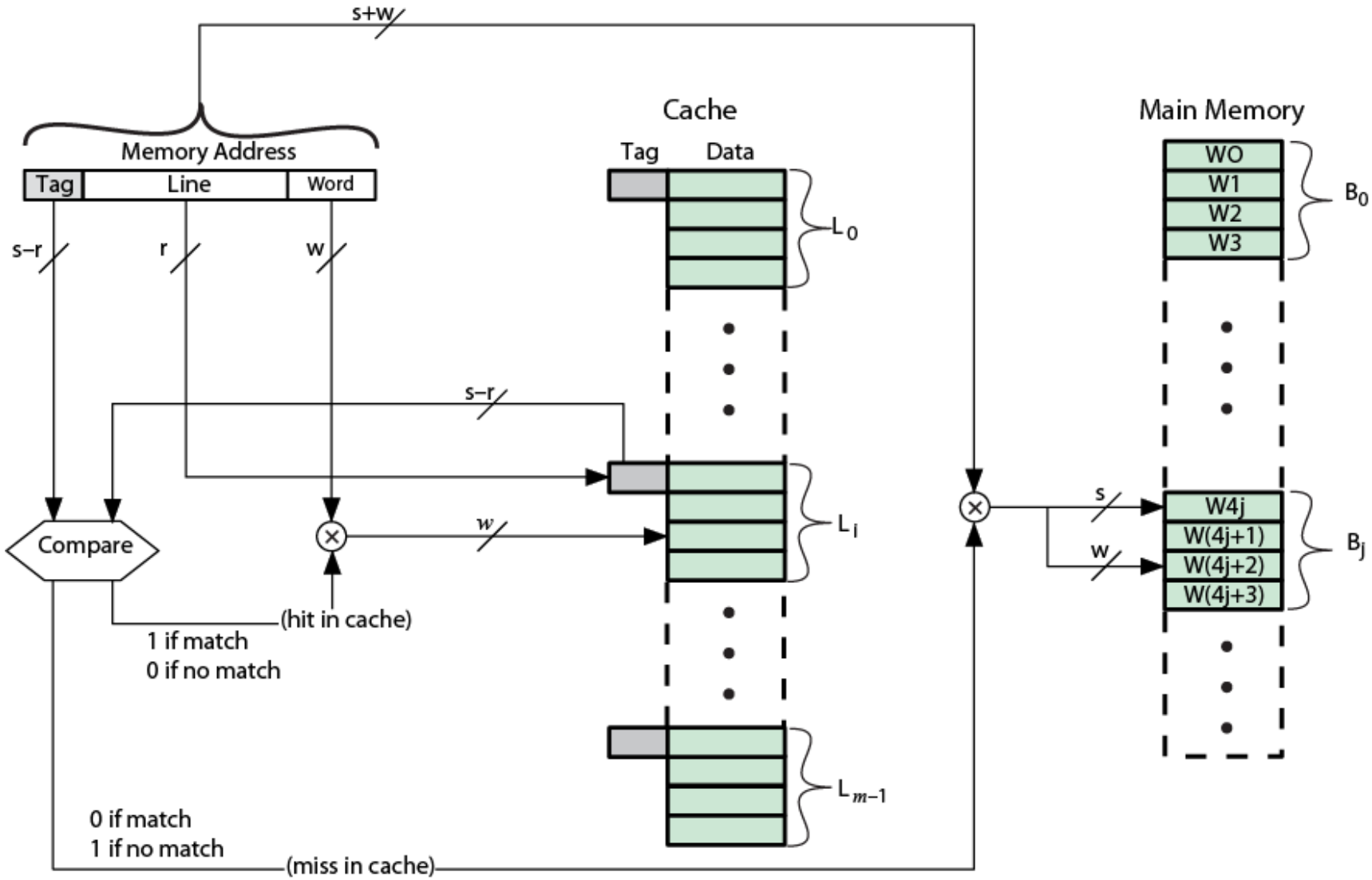
(b) Main memory


P. A. bits :
0 1 1 1 1 1
7
3
2^5 2^4 2^3 2^2 2^1 2^0
32 16 8 4 2 1
0 1 1 1 1 1
16 + 8 + 4 + 2 + 1 = 31
0 1 2 3
1 4 5 6 7
2 8 9 10 11
3 12 13 14 15
4 16 17 18 19
5 20 21 22 23
6 24 25 26 27
7 28 29 30 31
8 32 33 34 35
9 36 37 38 39
10 40 41 42 43
11 44 45 46 47
12 48 49 50 51
13 52 53 54 55
14 56 57 58 59
15 60 61 62 63

Direct Mapping from Cache to Main Memory

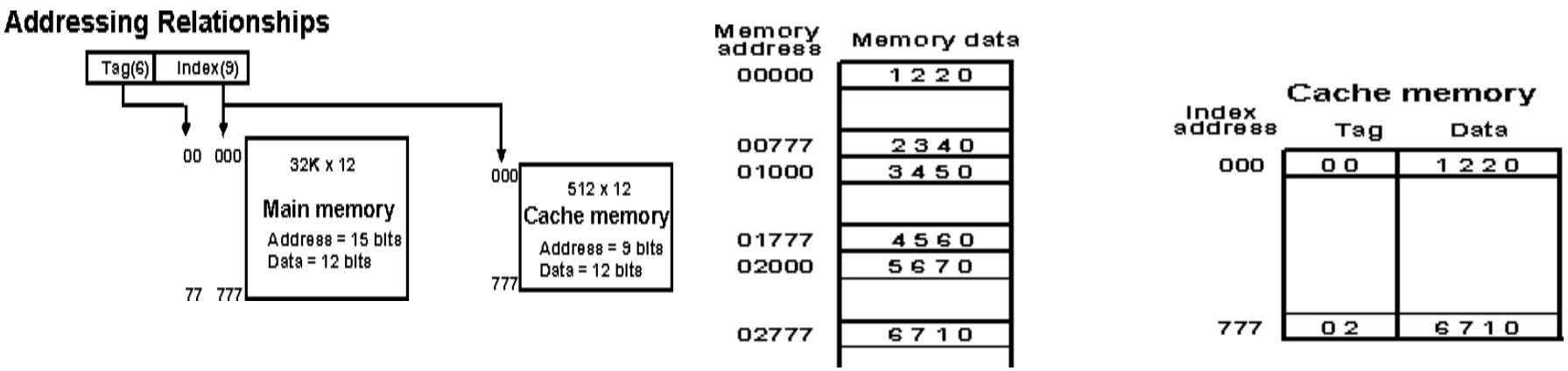


(a) Direct mapping


s+w
Memory Address
Tag
Line
Word
s−r
r
w
Cache
Tag
Data
L_0
L_i
L_{m-1}
s−r
w
Compare
(hit in cache)
1 if match
0 if no match
0 if match
1 if no match
(miss in cache)
Main Memory
WO
W1
W2
W3
B_0
s
w
W4j
W(4j+1)
W(4j+2)
W(4j+3)
B_j

Direct Mapping

- N-bit CPU memory address – k bits index field and (n-k) bits tag field
- Index bits are used to access the cache.
- Each word in cache consists of data word and its associated tag



On memory request, index field is used to access the cache.Tag field of CPU address is compared with the tag in the word read in the cache

Operation

CPU generates a memory request with (TAG;INDEX)

- - Access Cache using INDEX ; (tag; data)

Compare TAG and tag

- - If matches -> Hit

Provide Cache[INDEX](data) to CPU

- If not match -> Miss

-word is read from the memory

It is then stored in the cache together with the new tag replacing the previous value

- When memory is divided into blocks of words, index field – block field and word field
- Ex: 512 words cache – 64 blocks of 8words each – block field (6bits) and words field (3bits)

- Tags within the block are same.

Direct Mapping with block size of 8 words

	Index	tag	data
Block 0	000	0 1	3 4 5 0
	007	0 1	6 5 7 8
Block 1	010		
	017		
Block 63	770	0 2	
	777	0 2	6 7 1 0

6	6	3
Tag	Block	Word
	INDEX	

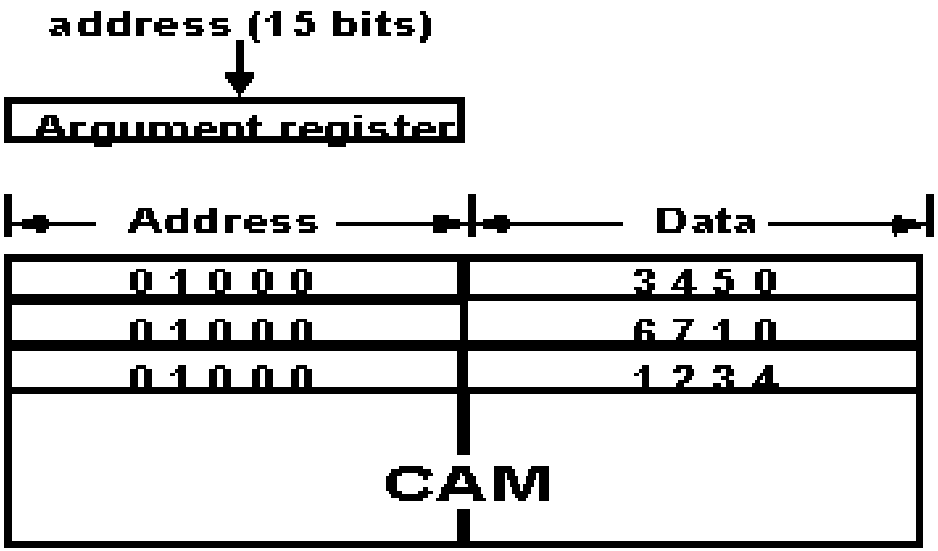
- When a miss occurs, entire block is transferred from main memory to cache.
- It is time consuming but improves hit ratio because of the sequential nature of programs.

Direct Mapping pros & cons

- Simple
- Inexpensive
- Fixed location for given block
  - If a program accesses 2 blocks that map to the same line repeatedly, cache misses are very high

Associative Mapping

- Fastest, most flexible but very expensive
- Any block location in cache can store any block in memory
- Stores both the address and the content of the memory word
- CPU address of 15 bits is placed in the argument register and the associative memory is searched for a matching address
- If found data is read and sent to the CPU else main memory is accessed.
- CAM – content addressable memory



Set-Associative Mapping

- Each word of cache can store 2 or more words of memory under the same index address

Set Associative Mapping Cache with set size of two

Index	Tag	Data	Tag	Data
000	0 1	3 4 5 0	0 2	5 6 7 0
777	0 2	6 7 1 0	0 0	2 3 4 0

The comparison logic is done by an associative search of the tags in the set similar to an associative memory search, thus the name "Set-Associative"

- Hit ratio increases as the set size increases but more complex comparison logic is required when number of bits in words of cache increases
- When a miss occurs and set is full, one of tag-data items are replaced using block replacement policy

Problems

- A set associative cache consists of 64 lines or slots, divided into four line sets. Main memory consists 4k blocks of 128 words each. Show the format of main memory addresses.

The cache is divided into 16 sets of 4 lines each. Therefore, 4 bits are needed to identify the set number. Main memory consists of $4K = 2^{12}$ blocks. Therefore, the set plus tag lengths must be 12 bits and therefore the tag length is 8 bits. Each block contains 128 words. Therefore, 7 bits are needed to specify the word.

	TAG	SET	WORD
Main memory address =	8	4	7

Performance analysis

- Look through: The cache is checked first for a hit, and if a miss occurs then the access to main memory is started.
- Look aside: access to main memory in parallel with the cache lookup;

- **Look through**

  TA = TC + (1-h)*TM

  TC is the average cache access time

  TM is the average main memory access time

  TA is the average access time.

- **Look aside**

  TA = h*TC + (1-h)*TM

- hit ratio $h = \dfrac{\text{number of references found in the cache}}{\text{total number of memory references}}$

- **Miss Ratio m=(1-h)**

Example: Assume that a computer system employs a cache with an access time of 20ns and a main memory with a cycle time of 200ns. Suppose that the hit ratio for reads is 90%,

a) what would be the average access time for reads if the cache is a "look-through"cache?

The average read access time (**TA**) = **TC + (1-h)*TM**

$$20ns + 0.10*200ns = 40ns$$

b) what would be the average access time for reads if the cache is a "look-Aside" cache?

The average read access time in this case (**TA**)

= **h*TC + (1-h)*TM**= $0.9*20ns + 0.10*200ns = 38ns$

Source of Cache misses

- **Compulsory Misses: (Cold start)**

  Occur when a program is first started

  Cache does not contain any of program's data yet

- **Capacity Misses:**

  Occurs because the cache has a limited size

  - (Working set is larger than cache size)
  - Solution: increase cache size

- **Conflict Misses:**

.

  - (One set fills up, but space in other cache sets)
  - Solution : increase cache size

Block Replacement

- **Least Recently Used: (LRU)**

  Replace that block in the set that has been in the cache longest with no reference to it.

- **First Come First Out: (FIFO)**

  Replace that block in the set that has been in the cache longest.

- **Least Frequently Used: (LFU)**

  Replace that block in the set that has experienced the fewest references

Update policies

- Write Through
- Write Back
- Write allocate
- No write allocate (Write Around)

Write through

- Update main memory with every memory write operation
- .

Advantage:

- easy to implement
- main memory always has the most current copy of the data (consistent)

Disadvantage:

- write is slower
- every write needs a main memory access
- as a result uses more memory bandwidth

Write back

- the information is written only to the block in the cache.The modified cache block is written to main memory only when it is replaced.
- To reduce the frequency of writing back blocks on replacement, a ***dirty bit*** is commonly used. This status bit indicates whether the block is dirty (modified while in the cache) or clean (not modified). If it is clean the block is not written on a miss.

Advantage:

- writes occur at the speed of the cache memory
- multiple writes within a block require only one write to main memory
- as a result uses less memory bandwidth

Disadvantage:

- harder to implement
- main memory is not always consistent with cache

Write allocate and Write around

- **Write-Around**
  - **correspond to items not currently in the cache (i.e. write misses)**
  - **the item could be updated in main memory only without affecting the cache.**

- **Write-Allocate**
  - **update the item in main memory and bring the block containing the updated item into the cache.**

A two-way set associative cache has lines of 16 bytes and a total size of 8k bytes. The 64-Mbyte main memory is byte addressable. Show the format of main memory addresses.

Problem 2

- A two-way set associative cache has lines of 16 bytes and a total size of 8k bytes. The 64-Mbyte main memory is byte addressable. Show the format of main memory addresses.

There are a total of 8 kbytes / 16 bytes = 512 lines in the cache. Thus the cache consists of 256 sets of 2 lines each. Therefore 8 bits are needed to identify the set number. For the 64-Mbyte main memory, a 26-bit address is needed. Main memory consists of 64-Mbyte / 16 bytes = 2^{22} blocks. Therefore, the set plus tag lengths must be 22 bits, so the tag length is 14 bits and the word field length is 4 bits.

	TAG	SET	WORD
Main memory address =	14	8	4

- Consider a memory system with T_c = 100ns and T_m = 1200ns. If the effective access time is 10% greater than the cache access time, what is the hit ratio H in look-through cache?

$\Rightarrow T_A = T_C + (1-h)*T_M$

$\Rightarrow 1.1\ T_c = T_c + (1-h)*T_M$

$\Rightarrow 0.1T_C = (1-h)*\ T_M$

$\Rightarrow 0.1 * 100 = (1-h) *1200$

$\Rightarrow 1-h = 10/1200$

$\Rightarrow h = 1190/1200$

A computer system employs a write-back cache with a 90% hit ratio for writes. The cache operates in "Look Through" and has a 80% read hit ratio. Reads account for 60% of all memory references and writes account for 40%.If the main memory cycle time is 300ns and the cache access time is 30ns, what would be the average access time for all references (reads as well as writes)?

TC	TM	Reads	Reads	Reads	Writes	Writes	Writes
ns	ns	%	Hit	Miss	%	Hit	Miss
30	300	60	80	20	40	90	10

Reads	Writes	Total
Look Through	Look Through	Look Through
Tc +(m*TM)=	Tc +(m*TM)=	
30+(0.2*300)= 90	30+(0.1*300) = 60	((90*0.6)+(60*0.4)) = 78

References

- J. L. Hennessy & D.A. Patterson, Computer architecture: A quantitative approach, Fourth Edition, Morgan Kaufman, 2004.